

AI-assisted position scaling

Architecture and training pipeline for conviction-based averaging-in

Design notes for improving win rate and risk-adjusted returns in an existing algorithmic trading platform (technical analysis, AI signal, market regime engine, decision engine, ML prediction, strategy optimizer, portfolio manager, paper trading, and backtesting modules already in place).

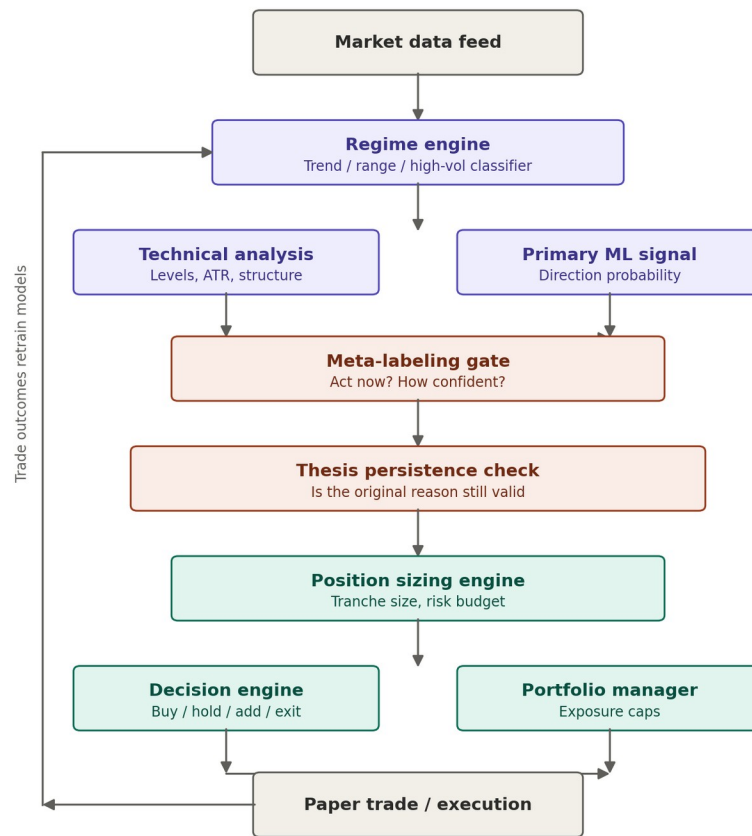
1. The core idea

Naive “average down when price drops” is a martingale: position size grows exactly as the trade gets worse, so a single regime-shift event can cause outsized losses. The goal instead is conviction-based position scaling — adding to a position only when both price has moved favorably for entry AND independent evidence still supports the original thesis.

This mirrors how a disciplined human trader operates: they don't buy more just because price fell, they buy more because price fell and their reasoning for owning the stock still holds. That “is it still worth it” check is the central design problem this document works through.

2. Architecture overview

The existing modules (technical analysis, AI signal, market regime engine, ML prediction) are treated as evidence sources feeding into a hierarchical gating pipeline, rather than independent votes that get averaged together. Each stage can only pass through what the stage before it approved — poor signals get filtered out early, before capital is committed, instead of being diluted into an average.



2.1 Why this ordering matters

The gate order is: regime → signal → meta-labeling → thesis persistence → sizing. Running sizing before meta-labeling (as most naive systems do) means positions get sized up before checking whether the add was actually justified — exactly the martingale trap. Gating in this order kills bad ideas cheaply, before capital is committed.

3. Component design

3.1 Regime engine

Sits at the top of the pipeline and acts as a hard filter. Outputs a confidence-weighted regime vector (e.g. 70% trending-up, 20% range-bound, 10% high-volatility) rather than a single label. Scaling-in rules differ completely by regime: permissive in range-bound conditions, restrictive in trending-down conditions. Because every downstream decision inherits this classification, it deserves the most backtesting rigor —

ideally an ensemble of regime detectors (HMM, volatility clustering, trend-strength indicator) voting together rather than one model.

3.2 Technical analysis + primary ML signal

Run in parallel and feed the meta-labeling gate as features rather than as final buy/sell decisions. Technical analysis contributes distance to support/resistance, ATR, and volume profile. The ML model contributes a direction probability and expected magnitude. Neither independently triggers a trade.

3.3 Meta-labeling gate — the most critical new component

A second model, trained separately from the primary signal model, that answers a different question. The primary model asks “will this go up?” The meta-model asks “given the primary model fired, should I actually act on it, and with how much size?” This is the component most directly responsible for improving win rate, because it encodes “is this drop worth buying” rather than “did price go down.”

Feature inputs: primary signal probability, signal confidence delta since last entry, current drawdown, regime vector, realized volatility percentile, volume anomaly score, correlation to sector move (idiosyncratic vs. systematic), days since last entry, existing position size as percent of portfolio. Output: an act probability and a suggested size multiplier.

3.4 Thesis persistence check

A rules-based circuit breaker, cheap to run, that sits alongside the meta-model rather than replacing it. At entry, the system stores a structured snapshot: what signals fired, what regime was active, what the sector/peers were doing. On every re-evaluation it recomputes that snapshot and diffs it against the original. If enough of the original conditions have flipped, adds are blocked and the position is flagged for possible exit, regardless of what the meta-model says. This catches thesis-invalidation events the meta-model wasn't trained on — e.g. the first time a name gets a guidance cut.

3.5 Position sizing engine

Takes the meta-model's size multiplier and applies portfolio-level constraints: maximum tranches per name (a cap of 3 is a reasonable starting point), maximum sector exposure, and volatility-adjusted base sizing (smaller base size in high-volatility regimes). A Kelly-fraction or volatility-targeting formula lives here.

3.6 Decision engine and portfolio manager

Thin orchestration layers. Most of the decision intelligence already happened upstream; their job is converting the sizing engine's output into concrete orders and enforcing portfolio-wide caps.

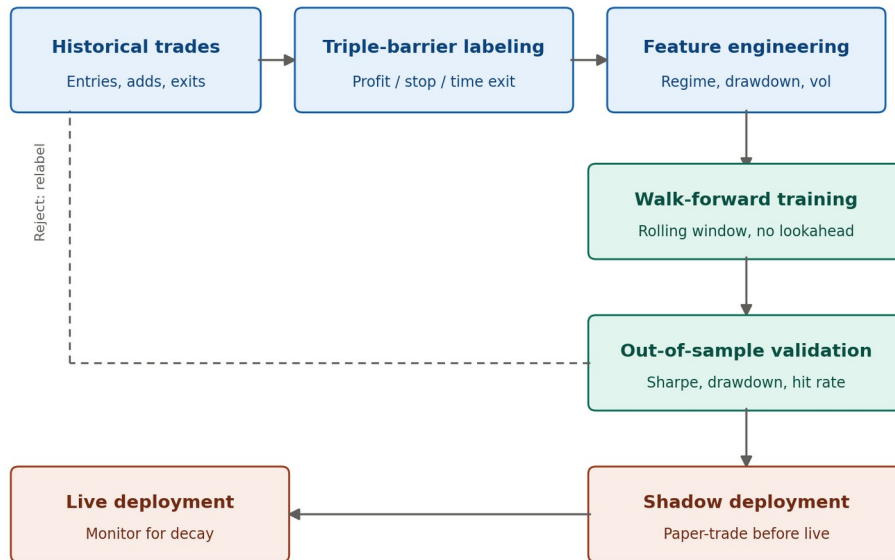
4. Criticality ranking

Components ranked by how much an error compounds downstream — not all pieces deserve equal engineering effort:

#	Component	Why it ranks here
1	Regime engine	An error here corrupts every decision below it. Warrants an ensemble approach and the most rigorous backtesting.
2	Meta-labeling gate	This is the actual win-rate lever. Retrain frequently and validate with walk-forward analysis, not a single train/test split.
3	Thesis persistence check	Less about model accuracy, more about feature completeness — a missing invalidation condition is worse than a noisy model.
4	Primary ML signal	Errors are partially absorbed by the meta-model, which is specifically trained to catch unreliable primary signals.
5	Position sizing math	Affects magnitude of gains/losses, not win/loss classification — more forgiving, but still needs hard caps as a backstop.

5. Meta-labeling training pipeline

The meta-labeling gate needs its own training pipeline, separate from the primary signal model:



5.1 Triple-barrier labeling

For every historical entry or add event, define three exit barriers — a profit target, a stop-loss, and a maximum holding time — expressed as multiples of ATR so they adapt per stock rather than using fixed percentages. Whichever barrier is hit first determines the label. This is extended to multi-tranche labeling: at each historical point where the system could have added to a position, label whether adding was the better choice versus holding versus exiting, based on what actually happened afterward.

5.2 Feature engineering

Regime, drawdown depth, volatility percentile, and thesis-condition diffs are computed as of that historical moment only — using any future information here is the single most common bug in these pipelines.

5.3 Walk-forward training

Never use a random train/test split on time-series trade data; it leaks future information into training. Rolling windows are used instead: train on months 1–12, validate on month 13, then slide forward. This gives a realistic estimate of performance on unseen data arriving in the order it actually would.

5.4 Out-of-sample validation

Evaluated on Sharpe ratio and maximum drawdown, not just accuracy — a model can have mediocre accuracy but still improve risk-adjusted returns by correctly sizing down bad trades. A failure here usually means barrier definitions or the feature set need adjustment, not just more data.

5.5 Shadow deployment

The new meta-model's decisions run in parallel with the live system for several weeks without being acted on, comparing what it would have done to actual outcomes, before it is allowed to touch position sizing.

6. Backtesting caveat

Scaling-in strategies are path-dependent: a position that adds on day 3 changes its average cost basis, which changes what “profit target hit” means for the whole position, which changes what the correct label is for day 3. The backtesting engine needs to simulate the full pipeline in sequence per historical bar and track multi-tranche cost basis, re-evaluating barriers per tranche — not just check entry/exit pairs. This is worth building before any meta-labeling results can be trusted.

Appendix A: triple_barrier_labeling.py

Generates the ground-truth training labels from historical trade events by simulating profit-target, stop-loss, and time-limit barriers.

```
"""
Triple-barrier labeling for meta-labeling training data.

For every historical entry/add event, this determines what the CORRECT
action would have been in hindsight: add, hold, or exit -- by simulating
three exit conditions (profit target, stop-loss, max holding time) and
seeing which one triggers first.

This is the ground-truth generator your meta-model trains against.
Reference: Lopez de Prado, "Advances in Financial Machine Learning", ch. 3.
"""

from dataclasses import dataclass
from datetime import datetime, timedelta
from enum import Enum
import numpy as np
import pandas as pd

class BarrierOutcome(Enum):
    PROFIT_TARGET = "profit_target"
    STOP_LOSS = "stop_loss"
    TIME_LIMIT = "time_limit"

@dataclass
class BarrierConfig:
    """
    Barriers are expressed as multiples of recent volatility (ATR or
    realized vol) rather than fixed percentages, so they adapt to each
    stock's own behavior instead of using one-size-fits-all thresholds.
    """
    profit_atr_multiple: float = 2.0
    stop_atr_multiple: float = 1.0
    max_holding_days: int = 20
    # if True, trailing stop tightens as profit_atr_multiple is approached
    use_trailing_stop: bool = False

@dataclass
class LabeledEvent:
    event_timestamp: pd.Timestamp
    symbol: str
    entry_price: float
    atr_at_entry: float
    outcome: BarrierOutcome
    exit_price: float
    exit_timestamp: pd.Timestamp
```



```

    holding_days: int
    realized_return: float
    # the actual training label: was this event a good moment to add size?
    label_add_was_correct: bool

def compute_barriers(entry_price: float, atr: float, config: BarrierConfig) -> tuple[float, float]:
    """Returns (upper_barrier_price, lower_barrier_price)."""
    upper = entry_price + config.profit_atr_multiple * atr
    lower = entry_price - config.stop_atr_multiple * atr
    return upper, lower

def label_single_event(
    price_path: pd.DataFrame, # columns: ['timestamp', 'high', 'low', 'close'], indexed from
    event forward
    entry_price: float,
    atr_at_entry: float,
    config: BarrierConfig,
) -> LabeledEvent:
    """
    Walk forward from the event day and find which barrier is hit first.
    price_path must already be sliced to start at (or just after) the event.
    """
    upper, lower = compute_barriers(entry_price, atr_at_entry, config)
    max_date = price_path["timestamp"].iloc[0] + timedelta(days=config.max_holding_days)

    window = price_path[price_path["timestamp"] <= max_date]

    for i, row in window.iterrows():
        if row["high"] >= upper:
            outcome = BarrierOutcome.PROFIT_TARGET
            exit_price = upper
            exit_ts = row["timestamp"]
            break
        if row["low"] <= lower:
            outcome = BarrierOutcome.STOP_LOSS
            exit_price = lower
            exit_ts = row["timestamp"]
            break
    else:
        # time barrier hit -- exit at last available close
        outcome = BarrierOutcome.TIME_LIMIT
        exit_price = window["close"].iloc[-1]
        exit_ts = window["timestamp"].iloc[-1]

    holding_days = (exit_ts - window["timestamp"].iloc[0]).days
    realized_return = (exit_price - entry_price) / entry_price

    # The label a meta-model actually needs: was adding size at this event
    # net positive? Profit-target hits are unambiguously correct; time-limit
    # exits are correct only if return exceeded a minimal threshold (avoids
    # rewarding flat, capital-tying trades); stop-losses are always wrong.

```

```

if outcome == BarrierOutcome.PROFIT_TARGET:
    label = True
elif outcome == BarrierOutcome.TIME_LIMIT:
    label = realized_return > 0.005 # tune this threshold per asset class
else:
    label = False

return LabeledEvent(
    event_timestamp=window["timestamp"].iloc[0],
    symbol="", # fill in by caller
    entry_price=entry_price,
    atr_at_entry=atr_at_entry,
    outcome=outcome,
    exit_price=exit_price,
    exit_timestamp=exit_ts,
    holding_days=holding_days,
    realized_return=realized_return,
    label_add_was_correct=label,
)

def build_labeled_dataset(
    events: pd.DataFrame, # columns: symbol, event_timestamp, entry_price, atr_at_entry
    price_history: dict[str, pd.DataFrame], # symbol -> OHLC dataframe
    config: BarrierConfig,
) -> pd.DataFrame:
    """
    Batch version: label every historical entry/add event in your trade log.
    This is the function your training pipeline calls to produce (X, y)
    pairs -- y comes from label_add_was_correct, X comes from the feature
    schema in meta_labeling.py.
    """
    labeled = []
    for _, event in events.iterrows():
        symbol_prices = price_history[event["symbol"]]
        path = symbol_prices[symbol_prices["timestamp"] >= event["event_timestamp"]]
        if path.empty:
            continue
        result = label_single_event(path, event["entry_price"], event["atr_at_entry"], config)
        result.symbol = event["symbol"]
        labeled.append(result)

    return pd.DataFrame([vars(r) for r in labeled])

```

Appendix B: thesis_persistence_gate.py

A rules-based circuit breaker that checks whether the original reasons for a trade still hold before allowing an add to a position.

```
"""
Thesis Persistence Gate.

At entry time, the system records WHY it entered -- a structured snapshot
of the conditions that justified the trade. Every time price moves against
an open position, this gate re-checks whether those same conditions still
hold. If the original thesis has broken, adding size is blocked regardless
of what the meta-model says -- this is a hard circuit breaker, not a
probabilistic decision.
"""

from dataclasses import dataclass, field
from enum import Enum
from typing import Optional
import pandas as pd

class RegimeLabel(Enum):
    TRENDING_UP = "trending_up"
    TRENDING_DOWN = "trending_down"
    RANGE_BOUND = "range_bound"
    HIGH_VOLATILITY = "high_volatility"

@dataclass
class ThesisSnapshot:
    """
    Captured once, at entry. Keep this small and specific -- every field
    here is something you're willing to say "if this changes, I was wrong."
    """
    symbol: str
    entry_timestamp: pd.Timestamp

    # what regime justified entering
    regime_at_entry: RegimeLabel
    regime_confidence_at_entry: float # 0-1

    # what the primary ML signal said
    signal_probability_at_entry: float
    signal_direction_at_entry: str # "long" or "short"

    # what technical structure justified the entry
    key_support_level: Optional[float] = None
    key_resistance_level: Optional[float] = None
    entry_price: float = 0.0

    # relative context -- was this idiosyncratic strength or a sector rally
```

```
sector_relative_strength_at_entry: float = 0.0 # stock return - sector return, trailing
20d
```

```
# free-form tags for fundamental/event-driven reasons, e.g. "earnings_beat",
"guidance_raise"
```

```
thesis_tags: list[str] = field(default_factory=list)
```

```
@dataclass
```

```
class ThesisCheckResult:
```

```
    conditions_checked: int
```

```
    conditions_broken: int
```

```
    broken_reasons: list[str]
```

```
    thesis_intact: bool
```

```
    recommendation: str # "allow_add", "hold_only", "consider_exit"
```

```
class ThesisPersistenceGate:
```

```
    """
```

```
    max_broken_conditions: how many of the original conditions are allowed
    to flip before the gate blocks further adds. Keep this low (1-2) --
    this component's value comes from being strict, not from being smart.
    """
```

```
    def __init__(self, max_broken_conditions: int = 1):
```

```
        self.max_broken_conditions = max_broken_conditions
```

```
    def check(
```

```
        self,
```

```
        snapshot: ThesisSnapshot,
```

```
        current_regime: RegimeLabel,
```

```
        current_regime_confidence: float,
```

```
        current_signal_probability: float,
```

```
        current_sector_relative_strength: float,
```

```
        current_price: float,
```

```
    ) -> ThesisCheckResult:
```

```
        broken = []
```

```
        # 1. regime flip -- the most common and most severe break
```

```
        if current_regime != snapshot.regime_at_entry:
```

```
            broken.append(
```

```
                f"regime changed from {snapshot.regime_at_entry.value} to
```

```
{current_regime.value}"
```

```
            )
```

```
        # 2. signal decay -- the model no longer believes in the direction
```

```
        signal_decay = snapshot.signal_probability_at_entry - current_signal_probability
```

```
        if signal_decay > 0.15: # tune per model calibration
```

```
            broken.append(
```

```
                f"signal probability decayed by {signal_decay:.2f}"
```

```
            )
```

```
        # 3. structural break -- price took out the level that justified entry
```

```
        if snapshot.key_support_level is not None:
```

```

        if snapshot.signal_direction_at_entry == "long" and current_price <
snapshot.key_support_level:
            broken.append("price broke below key support level used at entry")

        # 4. relative strength reversal -- stock-specific edge disappeared,
        # this is what distinguishes "the whole market dipped" (thesis fine)
        # from "this stock specifically is underperforming now" (thesis broken)
        rel_strength_delta = current_sector_relative_strength -
snapshot.sector_relative_strength_at_entry
        if snapshot.signal_direction_at_entry == "long" and rel_strength_delta < -0.05:
            broken.append("stock is now underperforming its sector, was outperforming at
entry")

        conditions_checked = 4
        conditions_broken = len(broken)
        thesis_intact = conditions_broken <= self.max_broken_conditions

        if thesis_intact and conditions_broken == 0:
            recommendation = "allow_add"
        elif thesis_intact:
            recommendation = "hold_only"
        else:
            recommendation = "consider_exit"

        return ThesisCheckResult(
            conditions_checked=conditions_checked,
            conditions_broken=conditions_broken,
            broken_reasons=broken,
            thesis_intact=thesis_intact,
            recommendation=recommendation,
        )

```

Appendix C: meta_labeling_gate.py

The trained classifier that decides whether to act on a primary signal and how much size to use, given the features assembled at each re-evaluation event.

```
"""
Meta-labeling gate: the model that decides WHETHER to act on a primary
signal and HOW MUCH size to use, given that a primary signal already fired.

This sits between your primary ML prediction model and your position
sizing engine. It is trained on the output of triple_barrier_labeling.py.
"""

from dataclasses import dataclass
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.calibration import CalibratedClassifierCV

FEATURE_COLUMNS = [
    "primary_signal_probability",
    "signal_confidence_delta",          # change in confidence since last entry/add
    "current_drawdown_pct",             # negative if underwater
    "regime_confidence",                 # how confident the regime engine is right now
    "regime_is_favorable",               # 1 if regime historically favors this trade type, else 0
    "realized_vol_percentile",           # trailing 20d realized vol, percentile vs 1yr history
    "volume_zscore",                     # volume anomaly vs 20d average
    "sector_correlation",                # how much of the move is sector-wide vs idiosyncratic
    "days_since_last_entry",
    "existing_position_pct_of_portfolio",
    "num_prior_adds",                   # tranche count so far
    "distance_to_support_atr",           # (price - support) / ATR
]

@dataclass
class MetaLabelPrediction:
    act_probability: float                # P(this is a good moment to act)
    suggested_size_multiplier: float      # 0.0 - 1.0, scales the base tranche size
    should_act: bool

class MetaLabelingGate:
    """
    Wraps a calibrated classifier. Calibration matters more here than in
    most classification tasks, because the raw probability IS the signal
    used downstream for sizing -- an uncalibrated model that's "usually
    right but overconfident" will systematically oversize positions.
    """

    def __init__(self, act_threshold: float = 0.55):
        self.act_threshold = act_threshold
```

```

        base_model = GradientBoostingClassifier(
            n_estimators=200,
            max_depth=3,          # shallow trees -- this dataset is not huge, avoid
overfitting
            learning_rate=0.05,
            subsample=0.8,
        )
        self.model = CalibratedClassifierCV(base_model, method="isotonic", cv=5)
        self._is_fitted = False

def fit(self, features: pd.DataFrame, labels: pd.Series):
    """
    features: columns matching FEATURE_COLUMNS
    labels: label_add_was_correct from triple_barrier_labeling.py

    IMPORTANT: split features/labels using walk-forward, not random
    shuffling, before calling this -- fit only on the training fold.
    """
    X = features[FEATURE_COLUMNS].values
    self.model.fit(X, labels.values)
    self._is_fitted = True

def predict(self, features: pd.Series) -> MetaLabelPrediction:
    if not self._is_fitted:
        raise RuntimeError("MetaLabelingGate must be fit before predicting")

    X = features[FEATURE_COLUMNS].values.reshape(1, -1)
    act_probability = float(self.model.predict_proba(X)[0, 1])

    # size multiplier scales smoothly with confidence above threshold,
    # rather than a binary on/off -- a 0.56 probability and a 0.95
    # probability should not get the same tranche size
    if act_probability < self.act_threshold:
        size_multiplier = 0.0
    else:
        span = 1.0 - self.act_threshold
        size_multiplier = min(1.0, (act_probability - self.act_threshold) / span)

    return MetaLabelPrediction(
        act_probability=act_probability,
        suggested_size_multiplier=round(size_multiplier, 3),
        should_act=act_probability >= self.act_threshold,
    )

def feature_importances(self) -> pd.Series:
    """
    Useful for sanity-checking the model: if 'current_drawdown_pct' is
    the dominant feature, the model has probably just re-learned naive
    averaging-down. You want regime and signal-decay features to carry
    real weight -- that's the difference this component is supposed to
    add over a price-only rule.
    """
    if not self._is_fitted:
        raise RuntimeError("Model must be fit first")

```

```

        # CalibratedClassifierCV wraps estimators per fold; average them
        importances = np.mean(
            [clf.estimator.feature_importances_ for clf in
self.model.calibrated_classifiers_],
            axis=0,
        )
        return pd.Series(importances, index=FEATURE_COLUMNS).sort_values(ascending=False)

def compute_features_for_event(
    primary_signal_probability: float,
    signal_probability_at_entry: float,
    current_price: float,
    entry_price: float,
    regime_confidence: float,
    regime_is_favorable: bool,
    realized_vol_percentile: float,
    volume_zscore: float,
    sector_correlation: float,
    days_since_last_entry: int,
    existing_position_pct_of_portfolio: float,
    num_prior_adds: int,
    support_level: float,
    atr: float,
) -> pd.Series:
    """
    Convenience function to assemble one feature row at inference time.
    Keep this in sync with FEATURE_COLUMNS above.
    """
    return pd.Series({
        "primary_signal_probability": primary_signal_probability,
        "signal_confidence_delta": primary_signal_probability - signal_probability_at_entry,
        "current_drawdown_pct": (current_price - entry_price) / entry_price,
        "regime_confidence": regime_confidence,
        "regime_is_favorable": float(regime_is_favorable),
        "realized_vol_percentile": realized_vol_percentile,
        "volume_zscore": volume_zscore,
        "sector_correlation": sector_correlation,
        "days_since_last_entry": days_since_last_entry,
        "existing_position_pct_of_portfolio": existing_position_pct_of_portfolio,
        "num_prior_adds": num_prior_adds,
        "distance_to_support_atr": (current_price - support_level) / atr if atr > 0 else 0.0,
    })

```